

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт

по лабораторной работе №2
по дисциплине **«Методы оптимизации»**

Автор: Беляева Валерия Андреевна

Факультет: ФПИиКТ

Группа: К3339

Поток: 1.3

Санкт-Петербург 2026

Лабораторная работа №2. Метод ломаных (метод Пиявского)

Задача. Найти глобальный минимум (максимум) липшицевой функции одной переменной $f(x)$ на отрезке $[a, b]$ с точностью ε по значению функции. На вход программа принимает строку с функцией, концы отрезка и ε ; на выходе — график функции, вспомогательных функций и итоговой ломаной, приближённые x^* и $f(x^*)$, число итераций и время.

Работа, как и ЛР1, выполнена в трёх видах: ручное прохождение метода, тот же расчёт в Excel (файл `excel_solution.xlsx`) и программа на Python (`python_solution.py`), которая демонстрируется на функциях Растригина и Экли.

Код программы: github.com/ValeriaBelyaeva/metopt/tree/main/lr2 (файл `python_solution.py`).

1. Метод

Функция липшицева на $[a, b]$ с константой L , если $|f(x) - f(y)| \leq L|x - y|$ для любых x, y . Тогда для каждой точки испытания x_i вспомогательная функция («зубец») ограничивает f снизу:

$$g_i(x) = f(x_i) - L|x - x_i| \leq f(x), \quad p_k(x) = \max_i g_i(x) \leq f(x).$$

Ломаная p_k — верхняя огибающая зубцов. Между соседними точками $x_i < x_{i+1}$ она состоит из двух звеньев $f(x_i) - L(x - x_i)$ и $f(x_{i+1}) + L(x - x_{i+1})$; приравнивая их, получаем нижнюю вершину:

$$\hat{x} = \frac{x_i + x_{i+1}}{2} + \frac{f(x_i) - f(x_{i+1})}{2L}, \quad \hat{p} = \frac{f(x_i) + f(x_{i+1})}{2} - \frac{L(x_{i+1} - x_i)}{2}.$$

Шаг метода: выбрать самую низкую нижнюю вершину $(\hat{x}, \hat{p})$ — это минимум ломаной; вычислить $f(\hat{x})$; вершина заменяется двумя новыми (слева и справа от $\hat{x}$), посчитанными по тем же формулам. Верхние вершины ломаной — точки испытаний $(x_i, f(x_i))$.

Остановка: пусть $f_k^* = \min_i f(x_i)$ — рекорд, $\hat{p}_k^*$ — минимум ломаной. Так как $\hat{p}_k^* \leq \min f \leq f_k^*$, при $f_k^* - \hat{p}_k^* \leq \varepsilon$ рекорд отличается от глобального минимума не более чем на ε . Максимум ищется как минимум функции $-f$.

2. Ручное прохождение метода

Пройдём первые шаги вручную для функции из задания $f(x) = x + \sin(3,14159x)$ на $[-3; 3]$, $\varepsilon = 0,01$. Производная $f'(x) = 1 + \pi \cos(\pi x)$, поэтому $|f'(x)| \leq 1 + \pi$ и берём $L = 1 + \pi \approx 4,14159$. Точки испытаний (верхние вершины) нумеруем так: $x_0 = a$, $x_1 = b$, далее x_{k+1} — точка k -й итерации.

Начало. $f(x_0) = f(-3) = -3.00001$, $f(x_1) = f(3) = 3.00001$. Рекорд $f^* = -3.00001$. Ломаная из двух звеньев $f(-3) - L(x + 3)$ и $f(3) + L(x - 3)$; их пересечение — нижняя вершина:

$$\hat{x} = \frac{-3 + 3}{2} + \frac{f(-3) - f(3)}{2L} = \frac{-3.00001 - 3.00001}{8,28318} = -0.72436, \quad \hat{p} = \frac{f(-3) + f(3)}{2} - \frac{L \cdot 6}{2} = -12.42477.$$

Итерация 1. Разность $f^* - \hat{p} = 9.42476 > \varepsilon$ — продолжаем. Испытание в вершине: $f(x_2) = f(-0.72436) = -1.48607$. Вершина заменяется двумя новыми — на $[x_0; x_2]$ и на $[x_2; x_1]$:

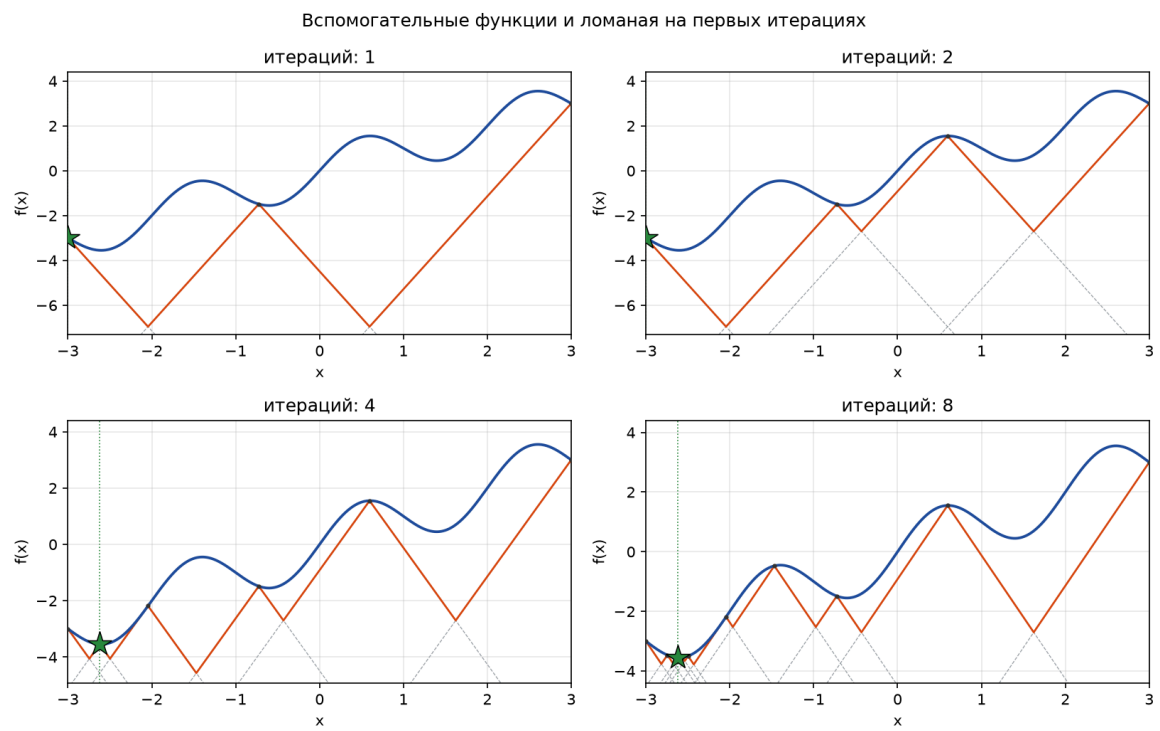
$$\begin{aligned} [-3; -0.7244] : \hat{p} &= \frac{-3.00001 + (-1.48607)}{2} - \frac{L(-0.72436 + 3)}{2} = -6.95542, \\ [-0.7244; 3] : \hat{p} &= \frac{-1.48607 + 3.00001}{2} - \frac{L(3 - (-0.72436))}{2} = -6.95542. \end{aligned}$$

Итерация 2. Обе вершины на одной высоте -6.95542 (функция почти нечётная) — можно взять любую, программа взяла правую: $\hat{x} = 0.59623$, $f(\hat{x}) = 1.55088$ — хуже рекорда, рекорд не меняется. **Итерация 3** — левая вершина: $\hat{x} = -2.04495$, $f = -2.18570$, новый рекорд. **Итерация 4:** $\hat{x} = -2.62078$, $f = -3.54965$ — метод уже нашёл окрестность глобального минимума и дальше уточняет её, лишь изредка проверяя другие участки, где ломаная ещё ниже рекорда.

k	интервал самой низкой вершины	$\hat{x}$	$\hat{p}$ (высота)	рекорд f^*	$f^* - \hat{p}$	$f(\hat{x})$ — испытание
1	$[-3.0000; 3.0000]$	-0.72436	-12.42477	-3.00001	9.42476	-1.48607
2	$[-0.7244; 3.0000]$	0.59623	-6.95542	-3.00001	3.95541	1.55088
3	$[-3.0000; -0.7244]$	-2.04495	-6.95542	-3.00001	3.95541	-2.18570
4	$[-3.0000; -2.0450]$	-2.62078	-4.57056	-3.00001	1.57055	-3.54965
5	$[-2.0450; -0.7244]$	-1.46912	-4.57056	-3.54965	1.02091	-0.47382
6	$[-3.0000; -2.6208]$	-2.74404	-4.06011	-3.54965	0.51045	-3.46427
7	$[-2.6208; -2.0450]$	-2.49753	-4.06011	-3.54965	0.51045	-3.49750
8	$[-2.6208; -2.4975]$	-2.56546	-3.77881	-3.54965	0.22915	-3.54439
...	...					
46	$[-2.6316; -2.6259]$	-2.62862	-3.55983	-3.55110	$0.00873 \leq 0.01$	стоп

Через 45 итераций самая низкая вершина ломаной -3.55983 отличается от рекорда меньше чем на ε :

$x^* \approx -2.60381, f(x^*) \approx -3.55110, \text{ гарантия: } f(x^*) - \min f \leq 0.00873.$



3. Расчёт в Excel

Лист «Метод ломаных» файла `excel_solution.xlsx` повторяет ручное прохождение до остановки. Параметры a, b, L, ε вынесены в ячейки B3:B6, $f(a), f(b)$ — B7:B8. Каждая итерация — строка таблицы: в неё вписан интервал самой низкой вершины (ссылки на ячейки уже найденных точек испытаний — это и есть «ручной» выбор шага), а всё остальное считается формулами:

Столбец	Формула (строка г)	Смысл
C–F	=B\$3, =K\$12, ...	$x_{\text{л}}, f(x_{\text{л}}), x_{\text{п}}, f(x_{\text{п}})$ — ссылки на точки испытаний
G	=(Cr+Er)/2+(Dr–Fr)/(2*B\$5)	абсцисса нижней вершины $\hat{x}$
H	=(Dr+Fr)/2–B\$5*(Er–Cr)/2	высота вершины $\hat{p}$
I	=MIN(B\$7;B\$8;K\$10:K(r–1))	рекорд f^*
J	=Ir–Hr	$f^* - \hat{p}$ — критерий остановки
K	=Gr+SIN(3.14159*Gr)	испытание $f(\hat{x})$
M	=IF(Jr<=B\$6;"СТОП...";"испытание")	проверка точности

Итог на листе: 45 испытаний, $x^* \approx -2.60381$ (поиск по INDEX/MATCH), $f(x^*) \approx -3.55110, f^* - \hat{p} = 0.00873 \leq 0,01$ — совпадает с ручным расчётом и программой. На листе «График» построены $f(x)$ и все точки испытаний: видно, как они сгущаются возле глобального минимума.

4. Программа

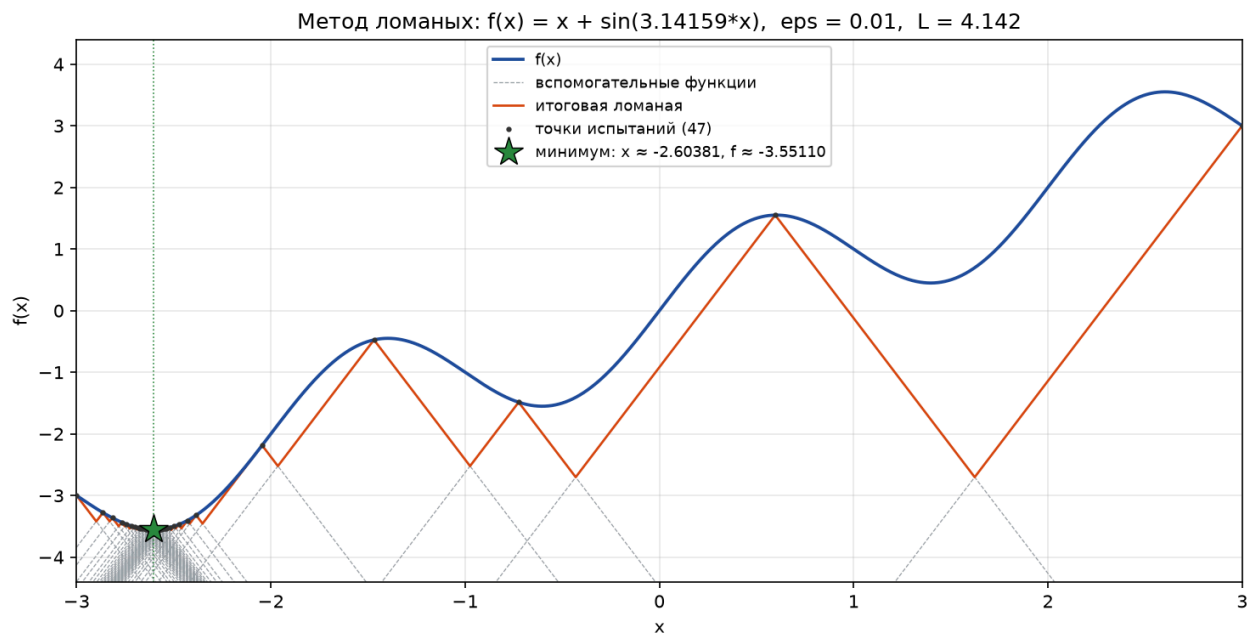
Программа `python_solution.py` (Python 3, numpy, matplotlib):

- `parse_function` — разбор строки вида $f(x) = x + \sin(3.14159 \cdot x)$ (допускаются $^$, π , e , abs , exp , тригонометрия); формула проверяется по белому списку, произвольный код не выполняется;
- `estimate_lipschitz` — оценка L по наклонам секущих с запасом 20%, если L не задана (сетка используется только для этой оценки и для рисования графика);
- `lower_vertex` — нижняя вершина ломаной на интервале (точка пересечения звеньев);
- `piyavskii` — сам метод: нижние вершины хранятся в двоичной куче по высоте, поэтому выбор самой низкой вершины стоит $O(\log k)$; ведётся история итераций; если точка функции оказалась ниже ломаной, программа сообщает, что L занижена;
- `plot_result`, `plot_steps` — график функции, вспомогательных функций, итоговой ломаной, точек испытаний и найденного экстремума;
- `solve`, `main` — ввод из командной строки или с клавиатуры, печать результатов, сохранение графика.

Запуск: `python3 python_solution.py "x + sin(3.14159*x)" -3 3 0,01` (ключи `--L`, `--max`), без аргументов — ввод с клавиатуры, `--demo` — все примеры ниже.

4.1. Пример из задания: $f(x) = x + \sin(3,14159x)$ на $[-3, 3]$, $L = 1 + \pi$

```
Функция:      f(x) = x + sin(3.14159*x)
Отрезок:      [-3, 3], eps = 0.01, ищем минимум
Константа Липшица: L = 4.14159 (задана)
Аргумент:     x* ≈ -2.60381247
Значение функции: f(x*) ≈ -3.55110191
Граница по ломаной: -3.55983368 (разность 8.732e-03 <= eps: True)
Число итераций: 45 (вычислений функции: 47)
Затраченное время: 0.073 мс
График:       figures/1_example.png
```

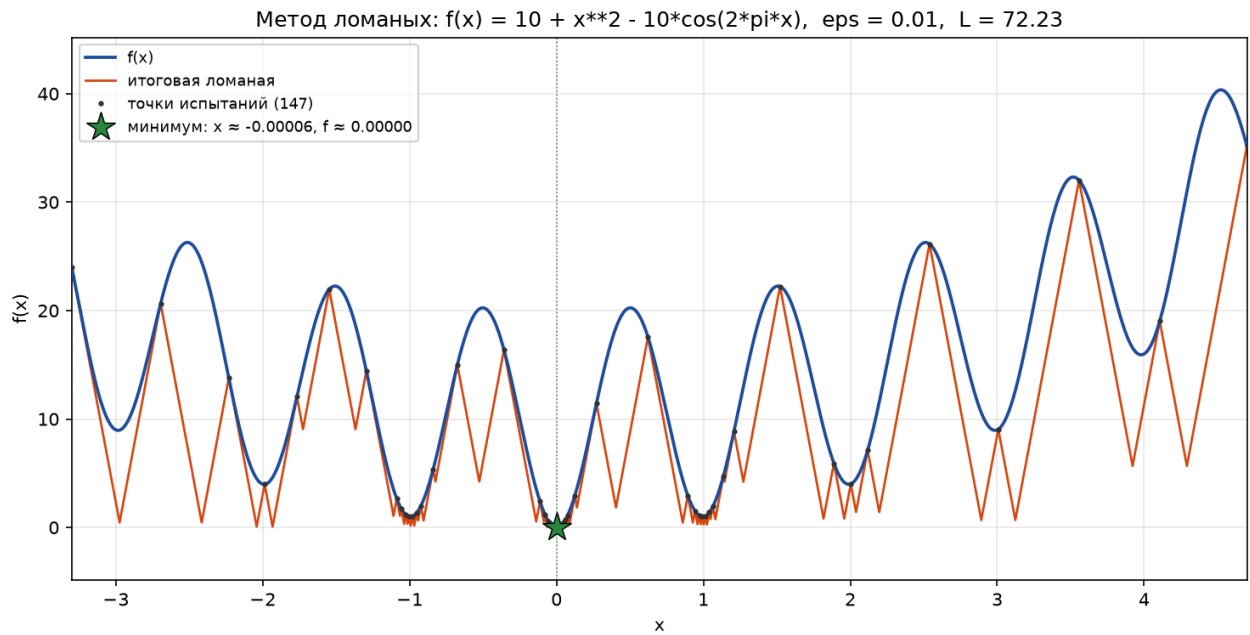


Метод ломаных для функции из задания: три локальных минимума, найден глобальный

4.2. Функция Растригина (одномерный срез)

$f(x) = 10 + x^2 - 10 \cos(2\pi x)$ на $[-3, 3; 4, 7]$ — восемь локальных минимумов, глобальный $f(0) = 0$ (отрезок несимметричный, чтобы метод не попал в $x = 0$ случайно). $|f'(x)| = |2x + 20\pi \sin(2\pi x)| \leq 9,4 + 20\pi$, поэтому $L = 9,4 + 20\pi \approx 72,23$.

```
Функция:      f(x) = 10 + x**2 - 10*cos(2*pi*x)
Отрезок:      [-3.3, 4.7], eps = 0.01, ищем минимум
Константа Липшица: L = 72.2319 (задана)
Аргумент:     x* ≈ -0.00006153
Значение функции: f(x*) ≈ 0.00000075
Граница по ломаной: -0.00986589 (разность 9.867e-03 <= eps: True)
Число итераций: 145 (вычислений функции: 147)
Затраченное время: 0.155 мс
График:       figures/2_rastrigin.png
```



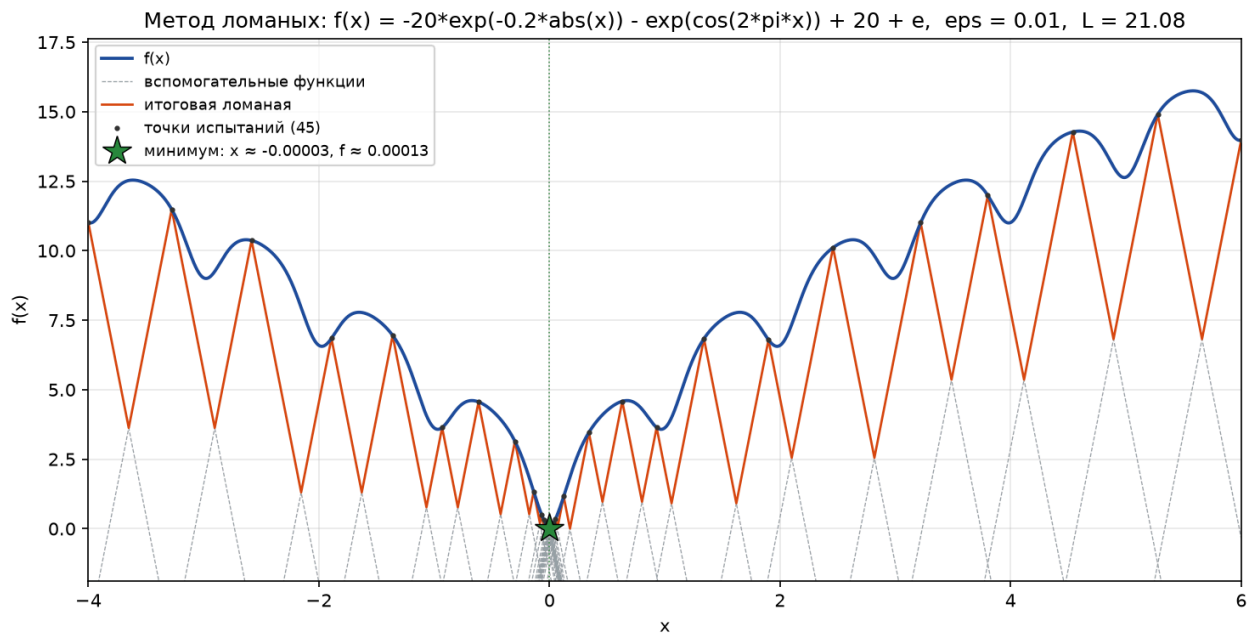
Функция Растригина: испытания сгущаются у глобального и у «конкурирующих» локальных минимумов

4.3. Функция Экли (одномерный срез)

$f(x) = -20e^{-0.2|x|} - e^{\cos 2\pi x} + 20 + e$ на $[-4, 6]$, глобальный минимум $f(0) = 0$; в точке минимума функция негладкая, но липшицева: $L = 4 + 2\pi e \approx 21.08$.

```

Функция:      f(x) = -20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e
Отрезок:      [-4, 6], eps = 0.01, ищем минимум
Константа Липшица: L = 21.0795 (задана)
Аргумент:     x* ≈ -0.00003345
Значение функции: f(x*) ≈ 0.00013388
Граница по ломаной: -0.00676951 (разность 6.903e-03 <= eps: True)
Число итераций: 43 (вычислений функции: 45)
Затраченное время: 0.062 мс
График:       figures/3_ackley.png
  
```



Функция Экли

4.4. Автоматическая оценка L , поиск минимума и максимума

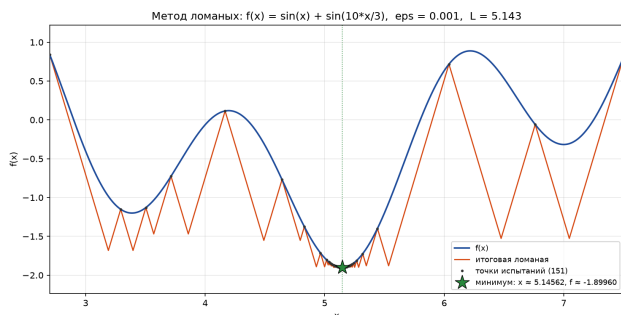
$f(x) = \sin x + \sin \frac{10x}{3}$ на $[2, 7; 7, 5]$, $\varepsilon = 0,001$; L не задаётся.

```

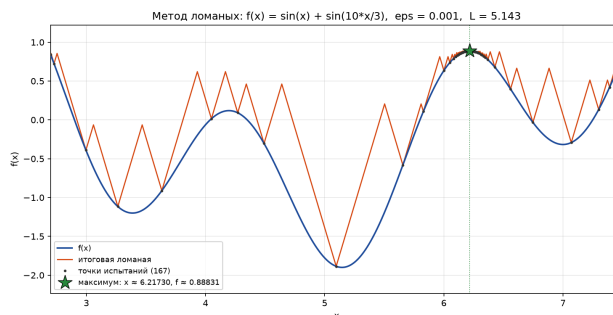
Функция:      f(x) = sin(x) + sin(10*x/3)
Отрезок:      [2.7, 7.5], eps = 0.001, ищем минимум
Константа Липшица: L = 5.14277 (оценка по сетке с запасом 20%)
Аргумент:     x* ≈ 5.14561857
Значение функции: f(x*) ≈ -1.89959927
Граница по ломаной: -1.90058331 (разность 9.840e-04 <= eps: True)
Число итераций: 149 (вычислений функции: 151)
Затраченное время: 0.227 мс
График:       figures/4_min.png

Функция:      f(x) = sin(x) + sin(10*x/3)
Отрезок:      [2.7, 7.5], eps = 0.001, ищем максимум
Константа Липшица: L = 5.14277 (оценка по сетке с запасом 20%)
Аргумент:     x* ≈ 6.21730159
Значение функции: f(x*) ≈ 0.88831478
Граница по ломаной: 0.88931097 (разность 9.962e-04 <= eps: True)
Число итераций: 165 (вычислений функции: 167)
Затраченное время: 0.233 мс
График:       figures/4_max.png

```



минимум



максимум (ломаная сверху)

4.5. Проверка и влияние точности

Для Растргина и Экли точный ответ известен ($x^* = 0$, $f^* = 0$): фактическая погрешность по значению $7,5 \cdot 10^{-7}$ и $1,3 \cdot 10^{-4}$ — меньше $\varepsilon = 0,01$. Зависимость числа итераций от ε (Растргин); для сравнения — сколько точек равномерной сетки дало бы ту же гарантию: $N = L(b - a)/(2\varepsilon)$:

eps	итераций	время, мс	x*	f(x*)	сетка с той же гарантией
0.1	69	0.085	+0.0001292	0.0000033	2890
0.01	145	0.156	-0.0000615	0.0000008	28893
0.001	389	0.396	+0.0000100	0.0000000	288928
0.0001	1219	1.285	-0.0000004	0.0000000	2889275
1e-05	3801	3.988	-0.0000001	0.0000000	28892742

Листинг python_solution.py

```

"""
LP2. Метод ломаных (метод Пиявского) — глобальный экстремум липшицевой функции на отрезке.
Беляева Валерия Андреевна, КЗ339, поток 1.3.

Запуск:
python3 python_solution.py          — ввод f(x), a, b, eps (и L) с клавиатуры
python3 python_solution.py "x + sin(3.14159*x)" -3 3 0.01 [--L 4.1416] [--max]
python3 python_solution.py --demo   — все демонстрации из отчёта (графики в figures/)

Экстремум ищется только методом ломаных: новые точки испытаний — аналитически вычисленные
нижние вершины ломаной (точки пересечения её звеньев), верхние вершины — точки испытаний.
Равномерная сетка используется лишь для рисования графика и (если L не задана) для оценки L.
"""
import argparse
import ast
import heapq
import time
from pathlib import Path

import numpy as np
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

FIG_DIR = Path(__file__).resolve().parent / "figures"

# ----- разбор строки f(x)

_NAMES = {name: getattr(np, name) for name in
["sin", "cos", "tan", "exp", "sqrt", "log", "log2", "log10",
"arcsin", "arccos", "arctan", "sinh", "cosh", "tanh", "floor", "ceil", "sign"]}
_NAMES.update({"abs": np.abs, "ln": np.log, "asin": np.arcsin, "acos": np.arccos,
"atan": np.arctan, "pi": np.pi, "e": np.e})
_NODES = (ast.Expression, ast.BinOp, ast.UnaryOp, ast.Call, ast.Name, ast.Load, ast.Constant,
ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow, ast.Mod, ast.USub, ast.UAdd)

def parse_function(text):
    """Строка 'f(x) = x + sin(3.14159*x)' -> функция f(x).
    Левую часть 'f(x) =' можно не писать, '^' понимается как степень.
    Формула проверяется по белому списку (никакого произвольного eval)."""

```

```

expr = text.strip()
if "=" in expr:
    expr = expr.split("=", 1)[1]
expr = expr.strip().replace("^", "**")
tree = ast.parse(expr, mode="eval")
for node in ast.walk(tree):
    if not isinstance(node, _NODES):
        raise ValueError(f"Недопустимая конструкция в формуле: {type(node).__name__}")
    if isinstance(node, ast.Name) and node.id != "x" and node.id not in _NAMES:
        raise ValueError(f"Неизвестное имя в формуле: {node.id}")
    if isinstance(node, ast.Call) and not isinstance(node.func, ast.Name):
        raise ValueError("Недопустимый вызов в формуле")
code = compile(tree, "<f(x)>", "eval")

def f(x):
    return eval(code, {"__builtins__": {}}, {**_NAMES, "x": x})

f.expr = expr
return f

def to_float(value):
    """Число или строка с числом; допускается десятичная запятая ('0,01')."""
    if isinstance(value, str):
        value = value.strip().replace(",", ".")
    return float(value)

# ----- константа Липшица

def estimate_lipschitz(f, a, b, n=2000, reserve=1.2):
    """Оценка L, если она не задана: max |Δf/Δx| по вспомогательной сетке с запасом 20%."""
    xs = np.linspace(a, b, n + 1)
    ys = np.asarray(f(xs), dtype=float) * np.ones_like(xs)
    slope = np.max(np.abs(np.diff(ys) / np.diff(xs)))
    return max(reserve * slope, 1e-12)

# ----- метод ломаных

def lower_vertex(xl, fl, xr, fr, L):
    """Нижняя вершина ломаной на [xl, xr] – пересечение прямых
    y = fl - L (x - xl) и y = fr + L (x - xr)."""
    xv = 0.5 * (xl + xr) + (fl - fr) / (2.0 * L)
    pv = 0.5 * (fl + fr) - 0.5 * L * (xr - xl)
    return pv, xv

def piyavskii(f, a, b, L, eps, mode="min", max_iter=1_000_000):
    """Глобальный экстремум липшицевой f на [a, b] методом ломаных.
    mode = 'min' или 'max' (максимум f = минимум -f).
    Возвращает словарь с ответом, историей итераций и данными для графика."""
    if not a < b:
        raise ValueError("Должно быть a < b")
    if L <= 0 or eps <= 0:
        raise ValueError("L и eps должны быть положительными")
    sign = 1.0 if mode == "min" else -1.0
    g = lambda x: sign * float(f(x))

    t0 = time.perf_counter()
    ga, gb = g(a), g(b)
    if abs(ga - gb) > L * (b - a):
        raise ValueError("Константа L занижена: условие Липшица нарушено на концах отрезка")
    trial_x, trial_g = [a, b], [ga, gb] # верхние вершины (точки испытаний)
    x_best, g_best = (a, ga) if ga <= gb else (b, gb)

    # куча нижних вершин: (высота, абсцисса, индекс левой точки, индекс правой точки)
    pv, xv = lower_vertex(a, ga, b, gb, L)
    heap = [(pv, xv, 0, 1)]
    history = []
    iterations = 0
    while True:
        pv, xv, il, ir = heap[0] # самая низкая вершина = минимум ломаной
        gap = g_best - pv
        if gap <= eps or iterations >= max_iter:
            history.append({"k": iterations + 1, "il": il, "ir": ir, "xv": xv, "pv": pv,
                           "record": g_best, "gap": gap, "stop": True})
            break
        heapq.heappop(heap)
        iterations += 1
        gv = g(xv) # новое испытание
        if gv < pv - 1e-9 * max(1.0, abs(pv)):
            raise ValueError(f"Константа L занижена: f({xv:.6g}) лежит ниже ломаной")
        history.append({"k": iterations, "il": il, "ir": ir, "xv": xv, "pv": pv, "gv": gv,
                       "record": g_best, "gap": gap, "stop": False})
        trial_x.append(xv)
        trial_g.append(gv)
        new = len(trial_x) - 1
        if gv < g_best:
            x_best, g_best = xv, gv
            # вершина заменяется двумя новыми – слева и справа от новой точки
            pl, xvl = lower_vertex(trial_x[il], trial_g[il], xv, gv, L)
            pr, xvr = lower_vertex(xv, gv, trial_x[ir], trial_g[ir], L)
            heapq.heappush(heap, (pl, xvl, il, new))
            heapq.heappush(heap, (pr, xvr, new, ir))
    elapsed = time.perf_counter() - t0

    return {
        "mode": mode, "a": a, "b": b, "L": L, "eps": eps, "sign": sign,
        "x": x_best, "f": sign * g_best, # приближённая точка и значение экстремума
        "bound": sign * heap[0][0], # гарантированная граница (минимум ломаной)
        "gap": g_best - heap[0][0], # достигнутая точность
        "iterations": iterations,
        "evaluations": len(trial_x),
        "time": elapsed,
    }

```

```

        "converged": g_best - heap[0][0] <= eps,
        "trial_x": np.array(trial_x),
        "trial_f": sign * np.array(trial_g),
        "vert_x": np.array([h[1] for h in heap]), # нижние вершины итоговой ломаной
        "vert_p": sign * np.array([h[0] for h in heap]),
        "history": history,
    }

# ----- визуализация

def broken_line(res):
    """Итоговая ломаная: чередование точек испытаний и нижних вершин."""
    x = np.concatenate([res["trial_x"], res["vert_x"]])
    y = np.concatenate([res["trial_f"], res["vert_p"]])
    order = np.argsort(x, kind="stable")
    return x[order], y[order]

def plot_result(f, res, title="", ax=None, max_aux=60, legend=True):
    a, b, L, sign = res["a"], res["b"], res["L"], res["sign"]
    own = ax is None
    if own:
        fig, ax = plt.subplots(figsize=(10, 5.2))
    xx = np.linspace(a, b, 2000) # сетка только для рисования
    yy = np.asarray(f(xx), dtype=float) * np.ones_like(xx)
    ax.plot(xx, yy, color="#1f4e9c", lw=1.8, label="f(x)", zorder=3)
    if len(res["trial_x"]) <= max_aux: # вспомогательные функции f(x_i) ± L|x - x_i|
        for i, (xi, fi) in enumerate(zip(res["trial_x"], res["trial_f"])):
            ax.plot(xx, fi - sign * L * np.abs(xx - xi), color="#9aa0a6", lw=0.6, ls="--",
                    label="вспомогательные функции" if i == 0 else None, zorder=1)
    bx, by = broken_line(res)
    ax.plot(bx, by, color="#d9480f", lw=1.3, label="итоговая ломаная", zorder=2)
    ax.plot(res["trial_x"], res["trial_f"], "", color="#333333", ms=4,
            label=f"точки испытаний ({res['evaluations']})", zorder=4)
    kind = "минимум" if res["mode"] == "min" else "максимум"
    ax.plot([res["x"]], [res["f"]], "k", color="#2b8a3e", ms=17, mec="black", mew=0.7, zorder=5,
            label=f"{kind}: x ≈ {res['x']:.5f}, f ≈ {res['f']:.5f}")
    ax.axvline(res["x"], color="#2b8a3e", lw=0.8, ls=":", zorder=1)
    span = max(yy.max() - yy.min(), 1e-9)
    lo, hi = yy.min() - 0.12 * span, yy.max() + 0.12 * span
    if res["mode"] == "min":
        lo = min(lo, max(by.min(), yy.min() - 0.6 * span) - 0.05 * span)
    else:
        hi = max(hi, min(by.max(), yy.max() + 0.6 * span) + 0.05 * span)
    ax.set_ylim(lo, hi)
    ax.set_xlim(a, b)
    ax.set_xlabel("x")
    ax.set_ylabel("f(x)")
    ax.set_title(title)
    ax.grid(alpha=0.3)
    if legend:
        ax.legend(loc="best", fontsize=8.5)
    if own:
        fig.tight_layout()
        return fig

def plot_steps(f, a, b, L, steps=(1, 2, 4, 8), mode="min"):
    """Как уточняется ломаная на первых итерациях."""
    fig, axes = plt.subplots(2, 2, figsize=(11, 7))
    for ax, k in zip(axes.ravel(), steps):
        res = piyavskii(f, a, b, L, eps=1e-12, mode=mode, max_iter=k)
        plot_result(f, res, title=f"итераций: {k}", ax=ax, legend=False)
    fig.suptitle("Вспомогательные функции и ломаная на первых итерациях")
    fig.tight_layout()
    return fig

# ----- запуск

def solve(func_str, a, b, eps, L=None, mode="min", fig_name=None):
    """Разбор функции, метод ломаных, печать результатов, график."""
    f = parse_function(func_str)
    a, b, eps = to_float(a), to_float(b), to_float(eps)
    auto = L is None
    L = estimate Lipschitz(f, a, b) if auto else to_float(L)
    res = piyavskii(f, a, b, L, eps, mode=mode)

    kind = "минимум" if mode == "min" else "максимум"
    print(f"Функция: f(x) = {f.expr}")
    print(f"Отрезок: [{a:g}, {b:g}], eps = {eps:g}, ищем {kind}")
    print(f"Константа Липшица: L = {L:.6g}" + (" (оценка по сетке с запасом 20%)" if auto else " (задана)"))
    print(f"Аргумент: x* ≈ {res['x']:.8f}")
    print(f"Значение функции: f(x*) ≈ {res['f']:.8f}")
    print(f"Граница по ломаной: {res['bound']:.8f} (разность {res['gap']:.3e} <= eps: {res['converged']})")
    print(f"Число итераций: {res['iterations']} (вычислений функции: {res['evaluations']})")
    print(f"Затраченное время: {res['time'] * 1000:.3f} мс")

    fig = plot_result(f, res, title=f"Метод ломаных: f(x) = {f.expr}, eps = {eps:g}, L = {L:.4g}")
    if fig_name:
        FIG_DIR.mkdir(exist_ok=True)
        fig.savefig(FIG_DIR / fig_name, dpi=150)
        print(f"График: figures/{fig_name}")
    plt.close(fig)
    print()
    return res

def demo():
    """Демонстрации из отчёта."""
    FIG_DIR.mkdir(exist_ok=True)
    print("=== 1. Пример из задания: f(x) = x + sin(3.14159x) на [-3, 3] ===")
    solve("f(x) = x + sin(3.14159*x)", -3, 3, "0.01", L=1 + 3.14159, fig_name="1_example.png")
    fig = plot_steps(parse_function("x + sin(3.14159*x)"), -3, 3, 1 + 3.14159)

```

```

fig.savefig(FIG_DIR / "1_steps.png", dpi=150)
plt.close(fig)

print("=== 2. Функция Растригина:  $10 + x^2 - 10\cos(2\pi x)$  на  $[-3.3, 4.7]$  ===")
L_rastrigin = 2 * 4.7 + 20 * np.pi
solve("10 + x^2 - 10*cos(2*pi*x)", -3.3, 4.7, 0.01, L=L_rastrigin, fig_name="2_rastrigin.png")

print("=== 3. Функция Экли:  $-20\exp(-0.2|x|) - \exp(\cos 2\pi x) + 20 + e$  на  $[-4, 6]$  ===")
L_ackley = 4 + 2 * np.pi * np.e
solve("-20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e", -4, 6, 0.01, L=L_ackley,
      fig_name="3_ackley.png")

print("=== 4.  $\sin(x) + \sin(10x/3)$  на  $[2.7, 7.5]$ : L оценивается автоматически, min и max ===")
solve("sin(x) + sin(10*x/3)", 2.7, 7.5, 0.001, fig_name="4_min.png")
solve("sin(x) + sin(10*x/3)", 2.7, 7.5, 0.001, mode="max", fig_name="4_max.png")

print("=== 5. Влияние eps (Растригин) ===")
f_r = parse_function("10 + x^2 - 10*cos(2*pi*x)")
print(f"{'eps':>8} | {'итераций':>9} | {'время, мс':>10} | {'x*':>11} | {'f(x*)':>11} | {'сетка с той же гарантией':>24}")
print("-" * 88)
for eps in (1e-1, 1e-2, 1e-3, 1e-4, 1e-5):
    r = piyavskii(f_r, -3.3, 4.7, L_rastrigin, eps)
    n_grid = int(np.ceil(L_rastrigin * 8.0 / (2 * eps)))
    print(f"{'eps':>8g} | {'iterations':>9d} | {'time':>10.3f} | {'x*':>11.7f} | "
          f"{'f':>11.7f} | {'n_grid':>24d}")

def main():
    p = argparse.ArgumentParser(description="Метод ломаных (Пиявского)")
    p.add_argument("func", nargs="?", help="функция, например 'x + sin(3.14159*x)'"")
    p.add_argument("a", nargs="?", help="a")
    p.add_argument("b", nargs="?", help="b")
    p.add_argument("eps", nargs="?", help="eps")
    p.add_argument("--L", help="константа Липшица (если не задана — оценивается)")
    p.add_argument("--max", action="store_true", help="искать максимум вместо минимума")
    p.add_argument("--demo", action="store_true", help="запустить демонстрацию из отчёта")
    args = p.parse_args()

    if args.demo:
        demo()
        return
    if args.func is None:
        # интерактивный ввод
        args.func = input("f(x) = ")
        args.a = input("a = ")
        args.b = input("b = ")
        args.eps = input("eps = ")
        args.L = input("L (Enter — оценить автоматически) = ") or None
    solve(args.func, args.a, args.b, args.eps, L=args.L, mode="max" if args.max else "min",
          fig_name="result.png")

if __name__ == "__main__":
    main()

```

5. Выводы

1. Реализован метод ломаных для поиска глобального минимума и максимума липшицевой функции одной переменной; функция задаётся строкой, результат — x^* , $f(x^*)$, гарантированная точность, число итераций, время и график.
2. Новые точки испытаний вычисляются аналитически как точки пересечения звеньев ломаной; нижние вершины хранятся в куче, верхние — в списке точек испытаний. Перебора отрезка по мелкой сетке для поиска экстремума нет.
3. Ручной расчёт, Excel и программа на функции из задания дают одинаковые 45 итераций и $x^* \approx -2.60381$, $f(x^*) \approx -3.55110$.
4. На функциях Растригина и Экли метод находит именно глобальный минимум среди нескольких локальных, с погрешностью меньше ε , и тратит на порядки меньше вычислений функции, чем равномерная сетка с той же гарантией.

Рефлексивный вывод

Самым наглядным оказалось ручное прохождение и таблица в Excel: после нескольких шагов видно, что метод не тратит испытания на участки, где ломаная уже выше рекорда, и сам «сбегается» к глобальному минимуму. Главная тонкость — константа Липшица: если её занизить, метод теряет гарантию и может пропустить минимум (поэтому в программе есть проверка, что точка функции не оказалась ниже ломаной), а если сильно завysить — зубцы становятся острее и итераций нужно больше. Пришлось разобраться, как получить L из оценки производной и как аккуратно хранить вершины, чтобы не перебирать отрезок, — за перебор по сетке в задании снимают баллы, и это хорошо показывает, в чём вообще идея метода.